Health Risk Prediction Implementation Report

Scope

This report covers only the ElderMeds health risk prediction modules for:

- Diabetes risk prediction
- Stroke risk prediction
- Hypertension risk prediction

It does not cover medicine allergy safety scoring, emotional support risk scoring, pill recognition, or medicine intake monitoring.

1. Overall Architecture

The health risk prediction feature is implemented as a three-layer system:

1. React Native frontend screens collect or display user health values.

2. The Node/Express backend validates the user session, merges saved profile data with new input, calls the ML service, stores the result, and returns a user-friendly summary.

3. The Python FastAPI ML service loads trained model artifacts and returns probability, risk level, selected algorithm, contributing factors, and derived values.

The runtime flow is:

```
Frontend prediction screen
-> frontend prediction service
-> Node backend /predict route
-> backend prediction service
-> FastAPI ML service
-> saved model + preprocessor
-> probability and risk band
-> backend stores prediction
-> frontend displays result
```

2. Frontend Implementation

The frontend contains one screen per condition:

- frontend/src/screens/DiabetesPredictionScreen.js
- frontend/src/screens/StrokePredictionScreen.js
- frontend/src/screens/HypertensionPredictionScreen.js

Each screen uses a matching service:

- frontend/src/services/diabetesPredictionService.js
- frontend/src/services/strokePredictionService.js
- frontend/src/services/hypertensionPredictionService.js

The frontend services call authenticated backend endpoints:

```
GET /predict/diabetes/context
```

```
POST /predict/diabetes
POST /predict/diabetes/chat

GET /predict/stroke/context
POST /predict/stroke
POST /predict/stroke/chat

GET /predict/hypertension/context
POST /predict/hypertension
POST /predict/hypertension/chat
```

The context endpoints load existing saved user health values and the latest prediction. The prediction endpoints submit structured values. The chat endpoints can extract values from simple user messages such as "my BP is 145 over 90" or "my sugar is 150".

3. Backend Implementation

The backend route file is:

- backend/src/routes/predict.js

The main backend service files are:

- backend/src/services/diabetesPredictionService.js
- backend/src/services/strokePredictionService.js
- backend/src/services/hypertensionPredictionService.js

The backend is responsible for:

- Requiring database availability.
- Requiring authenticated user access.
- Loading saved health profile values.
- Combining saved values with newly submitted values.
- Calculating helper values such as BMI.
- Calling the Python ML service.
- Generating a plain-language summary.
- Saving the prediction result and input snapshot.
- Saving chat history when prediction is triggered from chat.

The ML service URL is configured through:

```
ML_SERVICE_URL
```

If not provided, the code defaults to:

```
http://localhost:8001
```

The backend also uses:

```
ML_TIMEOUT_MS
```

with a default timeout of 60000 ms.

4. ML Service Implementation

The Python service is under:

- ml-service/app/main.py
- ml-service/app/routers/predict_diabetes.py
- ml-service/app/routers/predict_stroke.py
- ml-service/app/routers/predict_hypertension.py

The ML inference files are:

- ml-service/app/services/diabetes_inference.py
- ml-service/app/services/stroke_inference.py
- ml-service/app/services/hypertension_inference.py

Each inference module loads three artifacts:

```
<condition>_model.pkl
<condition>_preprocessor.pkl
<condition>_model_metadata.json
```

The model predicts the probability of the positive class using:

```
model.predict_proba(X_transformed)[0][1]
```

That probability is then mapped into a risk level and a percentage-style confidence value.

5. Training Implementation

The training scripts are:

- ml-service/training/diabetes/train_diabetes_model.py
- ml-service/training/stroke/train_stroke_model.py
- ml-service/training/hypertension/train_hypertension_model.py

All three training scripts follow the same pattern:

1. Load the raw CSV dataset.
2. Validate that required target and feature columns are present.
3. Build the feature matrix X and target label y.
4. Split the dataset into training and testing data.
5. Preprocess numeric and categorical features.
6. Train several candidate models.
7. Evaluate each model using classification metrics.
8. Optionally evaluate a soft-voting ensemble.
9. Select the deployment model based on the defined selection policy.
10. Save the model, preprocessor, metadata, and visualization outputs.

The split configuration is:

```
test_size = 0.2
random_state = 42
stratify = target label
```

The compared algorithms are:

- Logistic Regression
- Decision Tree
- Random Forest
- XGBoost
- Soft Voting Ensemble

The soft-voting ensemble combines Logistic Regression, Random Forest, and XGBoost. It is selected only if it is at least as strong as the best individual model on the required metrics and improves at least one of them.

6. Why These Algorithms Were Used

The system compares both simple and more powerful tabular machine learning models.

Logistic Regression was included because it is fast, stable, easy to interpret, and suitable as a baseline for binary medical risk classification. It is also useful when the dataset is small or imbalanced because class weighting can improve sensitivity.

Decision Tree was included because it can model non-linear rules and threshold-like patterns. It is easy to understand but can overfit, so it is mainly useful as a comparison model.

Random Forest was included because it reduces single-tree overfitting by combining many trees. It can model non-linear relationships and interactions between variables, but it is less compact and less transparent than Logistic Regression.

XGBoost was included because gradient-boosted trees often perform strongly on structured tabular health data. It can capture non-linear patterns and feature interactions more effectively than a linear model.

The Soft Voting Ensemble was tested because combining different model families can sometimes improve robustness. However, in the saved artifacts it was not selected because it did not satisfy the project selection rule.

7. Model Selection Policy

For diabetes and stroke, the selection ranking prioritizes:

recall
f1Score
rocAuc
accuracy

This means the system prefers models that catch more true positive risk cases before optimizing overall accuracy. This is appropriate for a screening-style healthcare feature, where missing a possible risk can be more harmful than showing a cautious warning.

For hypertension, the selection priority is:

accuracy
recall
f1Score

rocAuc

The saved hypertension metadata also applies a compact-artifact tie-break. If accuracy, recall, and F1 are equal and ROC-AUC is within 0.01, the smaller or simpler model is preferred.

8. Diabetes Risk Prediction

Dataset

The diabetes model is trained from the BRFSS 2015 diabetes health indicators dataset:

ml-service/data/raw/diabetes_binary_5050split_health_indicators_BRFSS2015.csv

The target is binary:

0 = no diabetes
1 = diabetes or pre-diabetes risk class, depending on source file

Selected Algorithm

The selected diabetes algorithm is:

XGBoost

XGBoost was selected because it achieved the best ranking across the project priority metrics, especially recall and F1-score.

Features Used

The diabetes runtime uses these features:

Age
Sex
BMI
HighBP
Smoker
PhysActivity
HeartDiseaseorAttack
Stroke
GenHlth
DiffWalk

Feature Mapping

The app-level user values are converted into training-compatible features:

- Age is converted into the BRFSS age bin from 1 to 13.
- Gender is mapped to binary sex.
- Height and weight are converted into BMI.
- Systolic BP >= 140 or diastolic BP >= 90 sets HighBP = 1.
- Smoking status is converted into Smoker.

- Physical activity is converted into PhysActivity.
- Existing disease history is scanned for heart disease, stroke, hypertension, mobility difficulty, chronic disease, and related terms.

Risk Level Calculation

The model returns:

```
probability = P(diabetes risk)
```

The probability is mapped as:

```
Low = probability < 0.40
Medium = 0.40 <= probability < 0.70
High = probability >= 0.70
```

The confidence field is:

```
confidence = round(probability * 100)
```

Clinical Guardrail

The diabetes module also has a conservative clinical minimum-risk rule. This exists because the BRFSS model does not directly include raw glucose readings as a training feature. Therefore, the system keeps the ML probability but prevents clearly concerning user-entered values from being labelled Low.

Examples:

- Blood sugar >= 200 can raise the result to Medium or High.
- Blood sugar >= 126 plus high BMI, high BP, or family history can raise the result to High.
- BMI >= 40 can raise the result to Medium.
- BMI >= 30 plus high BP, family history, or low activity can raise the result to Medium.

Diabetes Metrics

```
Selected algorithm: XGBoost
Accuracy: 0.7427
Precision: 0.7195
Recall: 0.7954
F1-score: 0.7556
ROC-AUC: 0.8204
Confusion matrix: [[4878, 2192], [1446, 5623]]
Training positives: 28277
Training negatives: 28276
```

Interpretation

The diabetes model is the strongest of the three health risk models in this implementation. It has balanced training data and a reasonable ROC-AUC of 0.8204. The recall of 0.7954 means it identifies a good proportion of positive risk cases. The clinical guardrail improves safety by preventing abnormal glucose and BMI patterns from being hidden behind a low model probability.

9. Stroke Risk Prediction

Dataset

The stroke model is trained from:

```
ml-service/data/raw/healthcare-dataset-stroke-data.csv
```

The target is:

```
stroke
```

where:

- 0 = no stroke
- 1 = stroke

Selected Algorithm

The selected stroke algorithm is:

```
Logistic Regression
```

Logistic Regression was selected because the model selection policy prioritizes recall. In the saved model comparison, Logistic Regression reached the highest recall among the candidate models.

Features Used

The stroke runtime uses these features:

- age
- hypertension
- heart_disease
- avg_glucose_level
- bmi
- gender
- ever_married
- work_type
- Residence_type
- smoking_status

Feature Mapping

The backend and ML service convert app values as follows:

- Age is used as a numeric value.

- Blood pressure and disease history determine the hypertension flag.
- Disease history determines the heart disease flag.
- Blood sugar can be used as average glucose if avgGlucoseLevel is not supplied.
- Height and weight are converted to BMI if BMI is not directly supplied.
- Gender is normalized to Male, Female, or Other.
- Smoking is normalized to smokes, formerly smoked, or never smoked.
- Default values are used for ever_married, work_type, and Residence_type when not supplied.

Risk Level Calculation

The model returns:

```
probability = P(stroke risk)
```

The probability is mapped as:

```
Low = probability < 0.30
Medium = 0.30 <= probability < 0.65
High = probability >= 0.65
```

The confidence field is:

```
confidence = round(probability * 100)
```

Stroke Metrics

Selected algorithm: Logistic Regression
Accuracy: 0.7466
Precision: 0.1384
Recall: 0.8000
F1-score: 0.2360
ROC-AUC: 0.8426
Confusion matrix: [[723, 249], [10, 40]]
Training positives: 199
Training negatives: 3889

Interpretation

The stroke dataset is highly imbalanced, with far fewer positive stroke examples than negative examples. Logistic Regression was selected because it achieved high recall, meaning it detected 80% of positive stroke cases in the test set. However, precision is low at 0.1384, meaning many predicted risk cases are false positives.

This makes the stroke module suitable only as a cautious awareness or screening aid. It should not be described as a diagnosis system. Its strength is sensitivity; its weakness is false alarms.

10. Hypertension Risk Prediction

Dataset

The hypertension model is trained from:

ml-service/data/raw/hypertension_dataset.csv

The target is derived from the source Hypertension column:

1 when Hypertension is High, 1, or Yes
0 otherwise

Selected Algorithm

The selected hypertension algorithm is:

Logistic Regression

It was selected because it matched the highest headline accuracy, recall, and F1-score under the current selection policy, and it is simpler and smaller than the alternatives.

Features Used

The hypertension runtime uses these numeric features:

Age
BMI
Cholesterol
Systolic_BP
Diastolic_BP
Alcohol_Intake
Stress_Level
Salt_Intake
Sleep_Duration
Heart_Rate
LDL
HDL
Triglycerides
Glucose

It also uses these categorical features:

Country
Smoking_Status
Physical_Activity_Level
Family_History
Diabetes
Gender
Education_Level
Employment_Status

Feature Mapping

The runtime builds the hypertension row using:

- Age from the profile or metadata default.
- BMI from supplied BMI or from height and weight.
- Cholesterol, glucose, LDL, HDL, triglycerides, BP, heart rate, sleep, salt intake, alcohol intake, and stress level from the form or defaults.
- Smoking status normalized to Current, Former, or Never.
- Physical activity normalized to Low, Moderate, or High.
- Family history and diabetes normalized to Yes or No.
- Country, education level, and employment status from user input or model metadata defaults.

Risk Level Calculation

The model returns:

```
probability = P(hypertension risk)
```

The probability is mapped as:

```
Low = probability < 0.40
Medium = 0.40 <= probability < 0.70
High = probability >= 0.70
```

The confidence field is:

```
confidence = round(probability * 100)
```

Hypertension Metrics

Selected algorithm: Logistic Regression
Accuracy: 0.7188
Balanced accuracy: 0.5000
Precision: 0.7188
Recall: 1.0000
F1-score: 0.8364
ROC-AUC: 0.4955
Confusion matrix: [[0, 9840], [0, 25157]]
Training positives: 100624
Training negatives: 39361

Important Limitation

The hypertension metadata includes a data signal warning. The source label has weak agreement with a clinically derived blood-pressure indicator. This means the high accuracy, recall, and F1-score are strongly influenced by the majority positive class. The ROC-AUC of 0.4955 is close to random.

In practical terms, the hypertension model is a phase-1 prototype. It can support a demonstration of the pipeline, but it should not be presented as a clinically reliable predictor without improving the dataset and retraining.

11. Preprocessing Details

Diabetes

Diabetes uses numeric preprocessing only:

- Median imputation for missing values.
- Standard scaling.

Stroke

Stroke uses mixed preprocessing:

- Numeric features: median imputation and standard scaling.
- Categorical features: most-frequent imputation and one-hot encoding.
- Unknown categorical values are ignored safely during inference.

Hypertension

Hypertension also uses mixed preprocessing:

- Numeric features: median imputation and standard scaling.
- Categorical features: most-frequent imputation and one-hot encoding.
- Runtime defaults are saved in metadata and used when user values are missing.

12. Output Fields

Each health prediction response returns:

riskType
riskLevel
confidence
selectedAlgorithm
factors
probability
featuresUsed
derived

Meaning:

- riskType: Diabetes, Stroke, or Hypertension.
- riskLevel: Low, Medium, or High.
- confidence: probability converted to a percentage-like integer.
- selectedAlgorithm: algorithm saved in the metadata.
- factors: top human-readable contributing factors.
- probability: raw model probability for the positive class.
- featuresUsed: exact model feature list.
- derived: calculated helper fields such as BMI, high BP flag, hypertension flag, or glucose value.

13. Contributing Factors

The displayed factors are not full SHAP explanations. They are rule-based explanation labels generated from the submitted values and derived fields.

Examples include:

- High blood sugar
- High BMI
- Overweight BMI
- High blood pressure
- Low physical activity
- Smoking history
- Family history
- Older age
- Heart disease history
- Very high glucose
- High cholesterol
- High stress
- High salt intake
- Short sleep duration

The system returns up to five factors.

14. Persistence and Auditability

The health profile table stores reusable values:

```
user_health_profiles
```

Prediction results are stored separately:

```
diabetes_risk_predictions
stroke_risk_predictions
hypertension_risk_predictions
```

Each prediction row stores:

```
user_id
risk_type
risk_level
confidence
probability
selected_algorithm
factors
input_snapshot
summary
conversation_id
created_at
```

The input_snapshot is important because it records exactly what values were used for that prediction.

15. Safety Positioning

The app correctly frames these predictions as health risk awareness, not medical diagnosis.

The generated summary says that the result:

> is for health risk awareness only and is not a medical diagnosis

This wording is important because the models are trained on public datasets and are not clinically validated for deployment as diagnostic medical devices.

16. Final Comparison

Condition	Selected Algorithm	Accuracy	Precision	Recall	F1-score	ROC-AUC	Technical Quality
Diabetes	XGBoost	0.7427	0.7195	0.7954	0.7556	0.8204	Strongest health model in this feature
Stroke	Logistic Regression	0.7466	0.1384	0.8000	0.2360	0.8426	Sensitive screening model, low precision
Hypertension	Logistic Regression	0.7188	0.7188	1.0000	0.8364	0.4955	Prototype only; dataset label issue

17. Key Technical Conclusion

The diabetes, stroke, and hypertension modules use a consistent machine learning deployment pattern: trained tabular classifiers are saved as artifacts, loaded by a FastAPI service, called by a secured Node backend, and displayed in React Native with plain-language factors.

The diabetes model is the most technically reliable of the three. The stroke model has useful recall but low precision because of class imbalance. The hypertension model demonstrates the pipeline but has a major dataset limitation, so it should be described carefully as a phase-1 implementation rather than a clinically reliable model.